

Lab 5: Simulating Confounding, Colliders, and DAGs

SOCIOL 723 — Social Statistics II — Thursday, September 24

Wenhao Jiang

Table of contents

1	Goals	1
2	Confounding	2
3	Colliders	3
3.1	Berkson’s paradox, directly	4
4	Post-treatment control	5
5	Selection bias	6
6	DAGs in R	7
6.1	Testable implications	8
7	Exercises	9
8	Session information	9

1 Goals

The point of this lab is that **you already know the right answer**. We build worlds where the causal effect is a number we chose, then see which analyses recover it and which do not. That is the only way to develop reliable intuition about identification.

1. Simulate confounding and watch adjustment fix it.
2. Simulate a collider and watch adjustment *break* a correct answer.
3. Simulate post-treatment (mediator) control.
4. See selection bias as collider bias.
5. Draw and query DAGs in R with `dagitty`.

```
library(dplyr)
library(ggplot2)
library(dagitty)
```

```
library(ggdag)
set.seed(723)
```

2 Confounding

The world: X causes both D and Y ; the true effect of D on Y is $\tau = 2$.

```
sim_confound <- function(n = 2000, tau = 2) {
  x <- rnorm(n) # confounder
  d <- 1.2 * x + rnorm(n) # treatment
  y <- tau * d + 1.5 * x + rnorm(n) # outcome
  data.frame(y, d, x)
}
dat <- sim_confound()

c(naive = coef(lm(y ~ d, data = dat))["d"],
  adjusted = coef(lm(y ~ d + x, data = dat))["d"],
  truth = 2)
#>      naive.d adjusted.d      truth
#>      2.755      1.998      2.000
```

The naive estimate is badly biased upward. Check the omitted variable bias formula from Week 1:

```
b_x <- coef(lm(y ~ d + x, data = dat))["x"] # effect of the omitted variable
delta <- coef(lm(x ~ d, data = dat))["d"] # omitted on included
c(predicted_bias = b_x * delta,
  actual_bias = coef(lm(y ~ d, data = dat))["d"] - coef(lm(y ~ d + x, data = dat))["d"])
#> predicted_bias.x      actual_bias.d
#>      0.7567      0.7567
```

Exactly as predicted. Over many replications the adjusted estimator is centred on the truth and the naive one is not:

```
reps <- t(replicate(1000, {
  d0 <- sim_confound()
  c(naive = coef(lm(y ~ d, d0))["d"], adjusted = coef(lm(y ~ d + x, d0))["d"])
}))

as.data.frame(reps) |>
  tidyr::pivot_longer(everything()) |>
  ggplot(aes(value, fill = name)) +
  geom_histogram(bins = 60, alpha = .7, position = "identity") +
  geom_vline(xintercept = 2, colour = "black", linetype = 2) +
```

```
scale_fill_manual(values = c("naive.d" = "firebrick", "adjusted.d" = "navy")) +
labs(x = expression(hat(tau)), y = "count", fill = NULL) +
theme_minimal(base_size = 10)
```

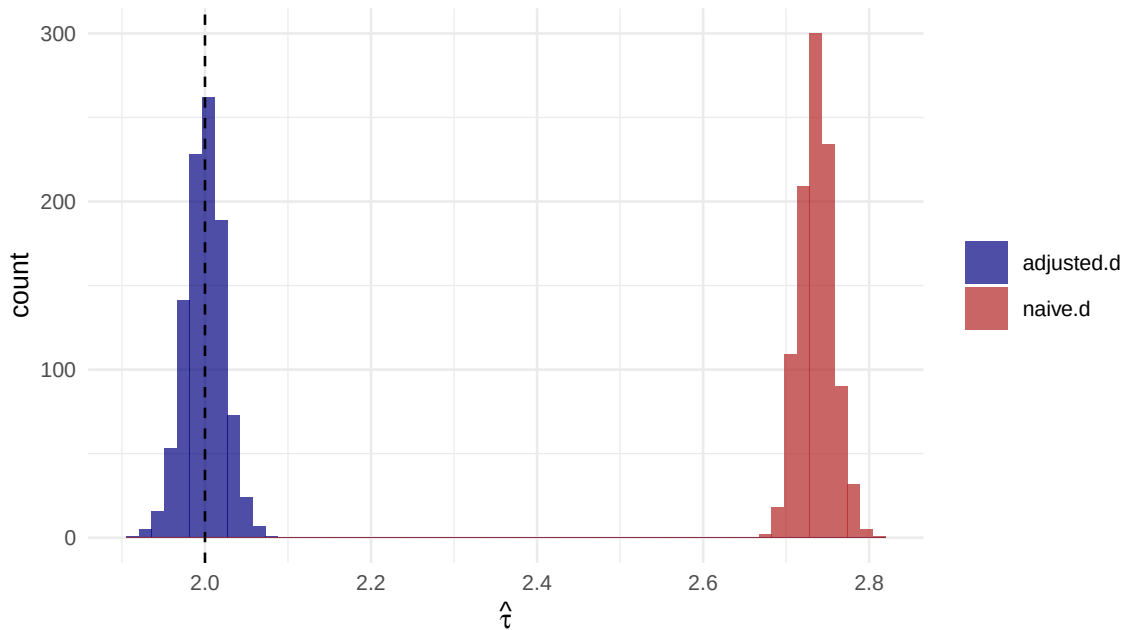


Figure 1: 1,000 replications. Adjustment recovers the true effect of 2; the naive estimator does not.

3 Colliders

Now the world has **no confounding at all**: D and Y share no common cause. But both cause C .

```
sim_collider <- function(n = 2000, tau = 2) {
  d <- rnorm(n)
  y <- tau * d + rnorm(n)
  c_ <- 1.0 * d + 1.0 * y + rnorm(n)    # collider: caused by BOTH
  data.frame(y, d, c = c_)
}
dat2 <- sim_collider()

c(correct = coef(lm(y ~ d, data = dat2))["d"],
  controlled = coef(lm(y ~ d + c, data = dat2))["d"],
  truth = 2)
#>   correct.d controlled.d      truth
#>    1.9345    0.4822    2.0000
```

! Important

Read that table again. The **unadjusted** regression is right; adding a control makes it wrong. Nothing about the data would tell you this. Only the DAG does.

```
reps2 <- t(replicate(1000, {
  d0 <- sim_collider()
  c(correct = coef(lm(y ~ d, d0))["d"], controlled = coef(lm(y ~ d + c, d0))["d"])
}))

as.data.frame(reps2) |>
  tidyr::pivot_longer(everything()) |>
  ggplot(aes(value, fill = name)) +
  geom_histogram(bins = 60, alpha = .7, position = "identity") +
  geom_vline(xintercept = 2, colour = "black", linetype = 2) +
  scale_fill_manual(values = c("correct.d" = "navy", "controlled.d" = "firebrick")) +
  labs(x = expression(hat(tau)), y = "count", fill = NULL) +
  theme_minimal(base_size = 10)
```

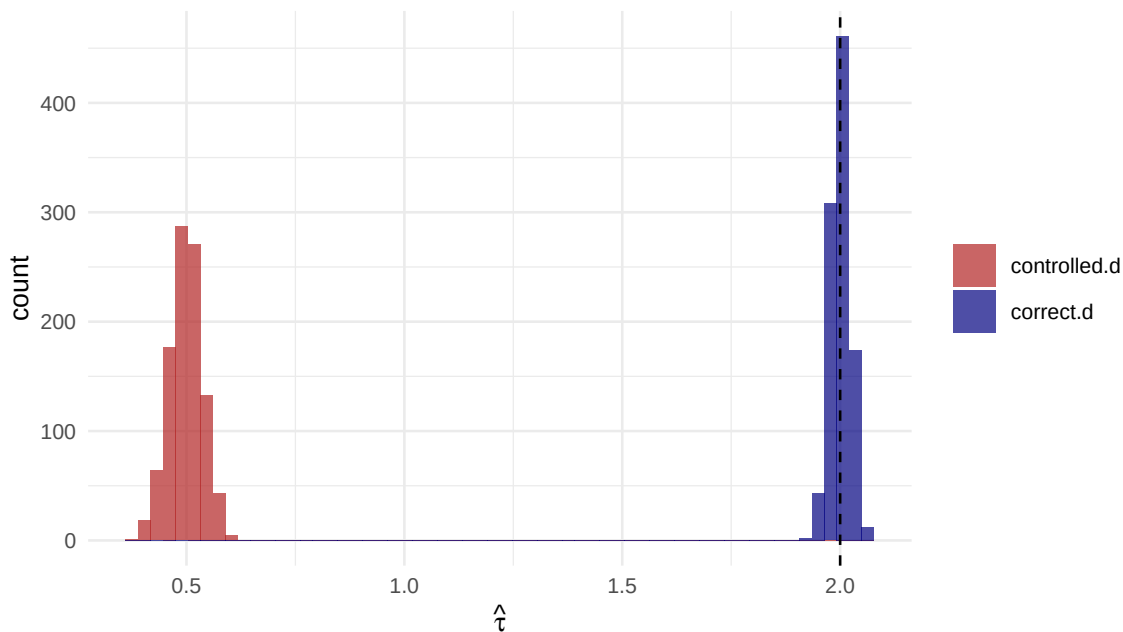


Figure 2: Controlling for a collider introduces bias where none existed.

3.1 Berkson's paradox, directly

Two genuinely independent traits become correlated once you condition on their common effect.

```
n <- 5000
test <- rnorm(n); athl <- rnorm(n)
admit <- (test + athl) > 1.5 # admitted if either is high enough
```

```
c(all_applicants = cor(test, athl),
  admitted_only = cor(test[admit], athl[admit]))
#> all_applicants admitted_only
#>      -0.004505      -0.698443
```

```
tibble::tibble(test, athl, admit) |>
  ggplot(aes(test, athl, colour = admit)) +
  geom_point(alpha = .25, size = .5) +
  geom_smooth(method = "lm", se = FALSE, linewidth = .7) +
  scale_colour_manual(values = c("grey70", "navy")) +
  labs(colour = "admitted") +
  theme_minimal(base_size = 10)
```

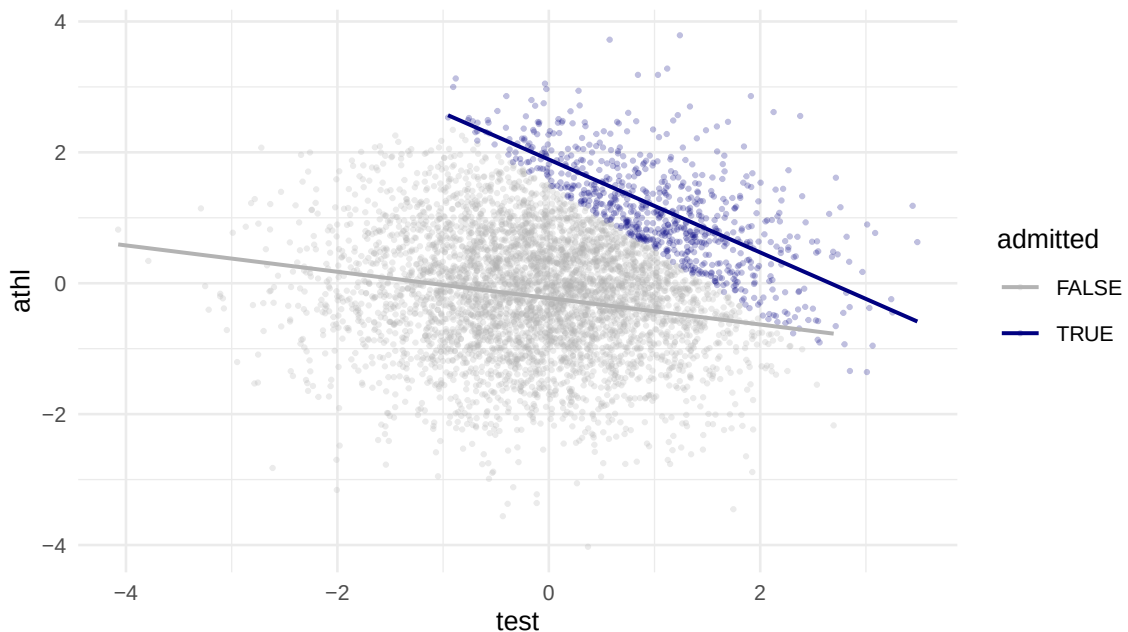


Figure 3: Test score and athletic talent are independent among applicants and negatively related among admitted students.

4 Post-treatment control

D affects Y both directly and through a mediator M . Controlling for M answers a different question.

```
sim_med <- function(n = 2000) {
  d <- rnorm(n)
  m <- 0.8 * d + rnorm(n)          # mediator
  y <- 1.0 * d + 1.5 * m + rnorm(n) # total effect = 1.0 + 0.8*1.5 = 2.2
  data.frame(y, d, m)
}
```

```

dat3 <- sim_med()

c(total_effect = coef(lm(y ~ d, data = dat3))["d"],
  direct_effect = coef(lm(y ~ d + m, data = dat3))["d"],
  truth_total = 2.2,
  truth_direct = 1.0)
#> total_effect.d direct_effect.d      truth_total      truth_direct
#>           2.204           1.056           2.200           1.000

```

Both numbers are “correct” — they estimate *different estimands*. The mistake is not computing the second; it is reporting it as though it were the first.

Warning

This is the single most common error in applied sociology: controlling for occupation when studying the effect of education on earnings, controlling for income when studying the effect of education on health, controlling for marriage when studying the effect of a policy on fertility.

5 Selection bias

Now the sample itself is selected on a collider. Only people who are employed appear in the data, and employment depends on both the treatment and the unobserved determinants of wages.

```

sim_select <- function(n = 5000, tau = 1) {
  educ <- rnorm(n)
  u <- rnorm(n) # unobserved ability
  wage <- tau * educ + u + rnorm(n)
  employed <- (0.8 * educ + 1.2 * u + rnorm(n)) > 0.5
  data.frame(wage, educ, employed)
}
d4 <- sim_select()

c(full_population = coef(lm(wage ~ educ, data = d4))["educ"],
  employed_only = coef(lm(wage ~ educ, data = d4, subset = employed))["educ"],
  truth = 1)
#> full_population.educ employed_only.educ      truth
#>           0.9940           0.7143           1.0000

```

The regression on the observed (employed) sample is biased **downward**, even though the specification is exactly the one that was unbiased in the full population. Restricting to the employed opens a path through the collider, so unobserved ability becomes correlated with education *within the sample*. The sample restriction *is* the conditioning.

6 DAGs in R

dagitty lets you state a graph and ask it what to condition on.

```
g <- dagitty('dag {  
  U [unobserved]  
  X -> D  
  X -> Y  
  U -> X  
  U -> C  
  D -> M  
  M -> Y  
  Y -> C  
  D [exposure]  
  Y [outcome]  
}')  
  
adjustmentSets(g, exposure = "D", outcome = "Y", effect = "total")  
#> { X }  
  
ggdag(tidy_dagitty(g, layout = "sugiyama"), text_size = 3,  
       node_size = 12) +  
  theme_dag_blank()
```

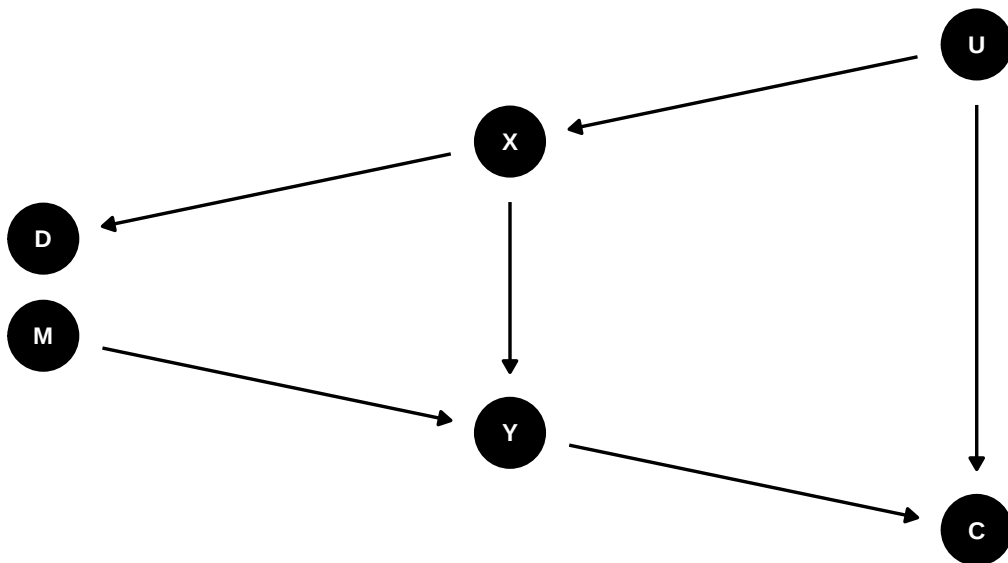


Figure 4: The DAG from lecture.

Ask it directly whether a candidate adjustment set works:

```
isAdjustmentSet(g, "X", exposure = "D", outcome = "Y")
#> [1] TRUE
isAdjustmentSet(g, c("X", "C"), exposure = "D", outcome = "Y") # collider!
#> [1] FALSE
isAdjustmentSet(g, c("X", "M"), exposure = "D", outcome = "Y") # mediator!
#> [1] FALSE
```

And which paths exist, open or blocked:

```
paths(g, from = "D", to = "Y")$paths |> head(6)
#> [1] "D -> M -> Y"          "D <- X -> Y"          "D <- X <- U -> C <- Y"
paths(g, from = "D", to = "Y", Z = "X")$open |> head(6)
#> [1] TRUE FALSE FALSE
```

6.1 Testable implications

A DAG implies conditional independencies. Those *are* testable, and they are the only part of a DAG that data can speak to.

```
impliedConditionalIndependencies(g) |> head(5)
#> C _||_ D | M, X
#> C _||_ D | X, Y
#> C _||_ M | X, Y
#> D _||_ Y | M, X
#> M _||_ X | D
```

```
## simulate from the DAG and check
sim <- simulateSEM(g, N = 5000)
localTests(g, sim, type = "cis.loess", R = 50) |> round(3) |> head(5)
#>           estimate std.error  2.5%  97.5%
#> C _||_ D | M, X   -0.003    0.015 -0.022  0.024
#> C _||_ D | X, Y   -0.011    0.016 -0.038  0.014
#> C _||_ M | X, Y   -0.036    0.011 -0.053 -0.011
#> D _||_ Y | M, X   -0.009    0.015 -0.038  0.019
#> M _||_ X | D       0.010    0.013 -0.019  0.033
```

i Note

A DAG that passes its testable implications is not thereby “correct” — many different DAGs imply the same independencies. But a DAG that *fails* them is wrong, and that is worth knowing.

7 Exercises

1. **Bad control that predicts treatment.** Simulate $Z \rightarrow D \rightarrow Y$ with Z having no effect on Y , plus an unobserved confounder $U \rightarrow D, U \rightarrow Y$. Compare the bias in $\hat{\tau}$ with and without controlling for Z . You should find that controlling for Z makes the bias *worse* — this is **bias amplification**. Explain why using the FWL logic from Week 1.
2. **Positivity.** Modify `sim_confound()` so that everyone with $x > 1$ is treated with probability 0.99. Estimate the ATE with and without those observations. What does regression do in the region with no comparison cases?
3. **Heterogeneous effects.** Make τ_i depend on x : $\tau_i = 1 + x_i$. Compute the true ATE and the true ATT by simulation, then see what `lm(y ~ d + x)` reports. Connect your answer to the variance-weighting result from Week 1.
4. **Your own DAG.** Draw the DAG for a causal question in your own research in `dagitty`. Mark unobserved variables. Ask for `adjustmentSets()`. Is there any set you can actually measure? If not, that is a finding — and it points you toward Weeks 7–11.
5. **The birth-weight paradox.** Simulate: smoking $\rightarrow$ birth weight, an unobserved severe condition $\rightarrow$ birth weight and $\rightarrow$ infant mortality, smoking $\rightarrow$ mortality (a genuine harmful effect). Show that among low-birth-weight infants, smoking appears protective.
6. **State an estimand.** For the GSS earnings data, write one sentence in the Lundberg–Johnson–Stewart form: a unit-specific quantity and a target population. Then say what would have to be true for `lm(lnearn ~ educ + ...)` to estimate it.

8 Session information

```
sessionInfo()
#> R version 4.4.1 (2024-06-14)
#> Platform: aarch64-apple-darwin20
#> Running under: macOS 26.5.2
#>
#> Matrix products: default
#> BLAS: /Library/Frameworks/R.framework/Versions/4.4-arm64/Resources/lib/libRblas.0.dylib
#> LAPACK: /Library/Frameworks/R.framework/Versions/4.4-arm64/Resources/lib/libRlapack.dylib;
#>
#> locale:
#> [1] C.UTF-8/C.UTF-8/C.UTF-8/C/C.UTF-8/C.UTF-8
#>
#> time zone: Asia/Shanghai
#> tzcode source: internal
#>
#> attached base packages:
```

```

#> [1] stats      graphics  grDevices  utils      datasets  methods   base
#>
#> other attached packages:
#> [1] ggdag_0.2.13  dagitty_0.3-4 ggplot2_4.0.0 dplyr_1.1.4
#>
#> loaded via a namespace (and not attached):
#> [1] viridis_0.6.5      utf8_1.2.4          generics_0.1.3      tidyr_1.3.1
#> [5] lattice_0.22-7     digest_0.6.37       magrittr_2.0.3      evaluate_1.0.0
#> [9] grid_4.4.1         RColorBrewer_1.1-3 fastmap_1.2.0       jsonlite_1.8.9
#> [13] Matrix_1.7-0       ggrepel_0.9.6       tinytex_0.53        gridExtra_2.3
#> [17] mgcv_1.9-1         purrr_1.0.2         fansi_1.0.6         viridisLite_0.4.2
#> [21] scales_1.4.0       tweenr_2.0.3        cli_3.6.5           rlang_1.3.0
#> [25] graphlayouts_1.2.2 polyclip_1.10-7     tidygraph_1.3.1     splines_4.4.1
#> [29] cachem_1.1.0       withr_3.0.1         yaml_2.3.10         tools_4.4.1
#> [33] memoise_2.0.1      boot_1.3-30         curl_5.2.3          vctrs_0.6.5
#> [37] R6_2.5.1           lifecycle_1.0.4     V8_7.0.0            MASS_7.3-60.2
#> [41] ggraph_2.2.2       pkgconfig_2.0.3     pillar_1.9.0        gtable_0.3.6
#> [45] glue_1.7.0         Rcpp_1.1.1          ggforce_0.5.0       xfun_0.47
#> [49] tibble_3.2.1       tidyselect_1.2.1    knitr_1.48          farver_2.1.2
#> [53] htmltools_0.5.8.1 nlme_3.1-164        igraph_2.1.4        rmarkdown_2.28
#> [57] labeling_0.4.3     compiler_4.4.1      S7_0.2.0

```